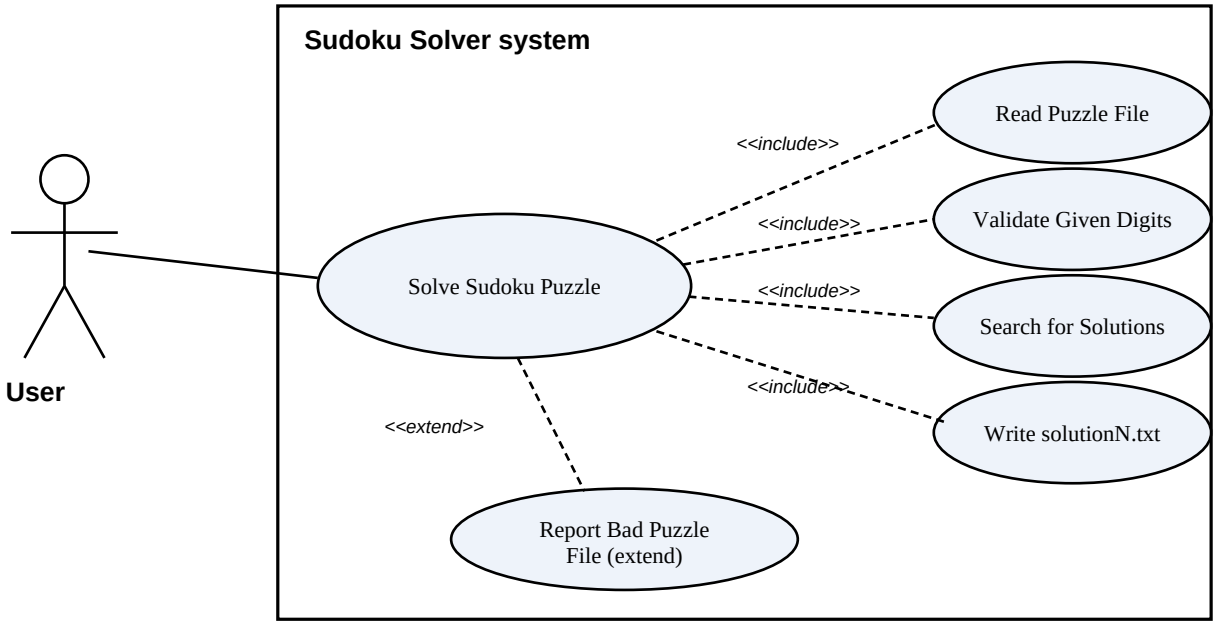
Assignment 5 Use Case Diagram

Hunter Noravong – EECS 348 – Revised 10/28/2026. Original Assignment 4 use case by Cesar Olono.



1. Use case identified

Solve Sudoku Puzzle (extended in Assignment 5 to write results to solution files).

2. Brief description

The user runs the program. For each of puzzle1.txt through puzzle5.txt, the program reads the puzzle, checks the given digits, searches for every solution with a recursive backtracking search, and writes the solutions (or "No solution found.") to the matching solutionN.txt file.

3. Step-by-step flow

1. The user runs the program.
2. The program builds the file names puzzleN.txt and solutionN.txt for N = 1.
3. The program creates (or overwrites) solutionN.txt.
4. The program reads puzzleN.txt into a 9x9 grid, recording which digits each row, column and 3x3 box already use.
5. The program checks that the given digits do not break any Sudoku rule.
6. The program picks the empty cell with the fewest legal digits and tries each legal digit in turn, backing up when stuck.
7. Each time the grid is full, the program writes that solution to solutionN.txt.
8. When the search ends, the program writes "Solutions found: n", or writes "No solution found." if there were none.
9. The program prints a one-line progress message to the screen.
10. The program repeats steps 2 to 9 for N = 2 through 5.

4. Detailed description

Use case name	Solve Sudoku Puzzle
Actor	User (runs the program from the command line)
Goal	Get every solution for each of the five puzzle files saved in solution1.txt through solution5.txt.
Preconditions	puzzle1.txt through puzzle5.txt are in the folder the program is run from. Each file has 81 cells: digits 1-9 and "_" for blanks, with spaces/newlines between cells. The folder allows the program to create files.
Main flow	Steps 1 to 10 above.
Included use cases	Read Puzzle File; Validate Given Digits; Search for Solutions; Write solutionN.txt.
Alternate flow A	puzzleN.txt cannot be opened or is not a valid puzzle (wrong characters, or not exactly 81 cells). The program prints an error on screen, writes "No solution found." to solutionN.txt, and moves on to the next puzzle.
Alternate flow B	The given digits already break a rule (for example two 5s in one row). No search is run, and solutionN.txt contains the single line "No solution found."
Alternate flow C	The puzzle has more than one solution. The search continues after each solution and writes all of them, separated by a blank line, followed by "Solutions found: n".
Alternate flow D	solutionN.txt cannot be created. The program prints an error on screen and moves on to the next puzzle.
Postconditions	For every readable puzzle, solutionN.txt exists and holds either all solutions in the same format as the puzzle files followed by "Solutions found: n", or the single line "No solution found."
Notes	Digit use is tracked with bitmasks so legality checks take constant time. The search always picks the empty cell with the fewest legal digits.